CS6650 Assignment 1: SkiTrak

Name: Yufan Wang

Student ID: 002504401

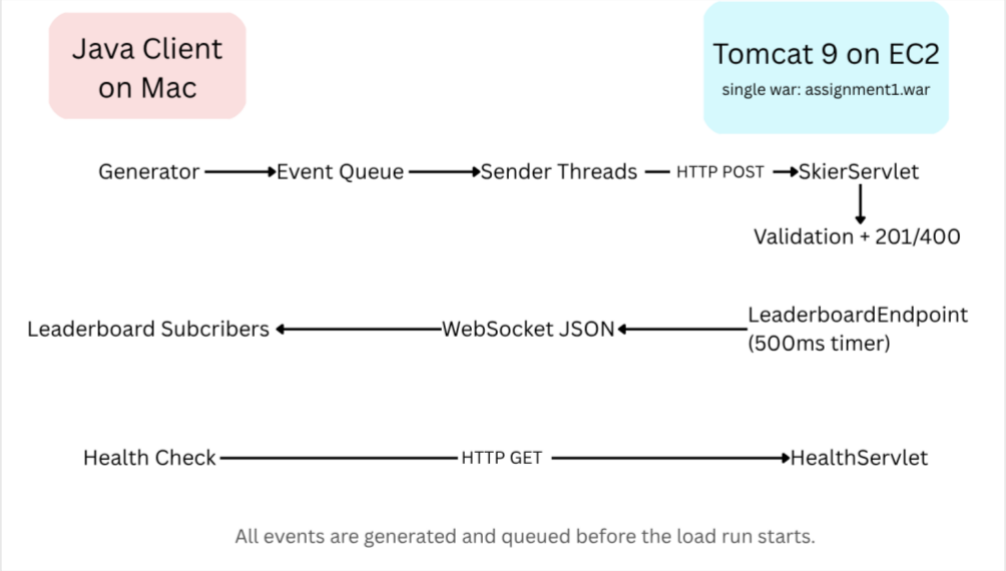
Git Repository: https://github.khoury.northeastern.edu/yufanwang/cs6650-assignment1.git

Test Date: Oct 16, 2026

Design Document

Architecture

My system has a Java load client and a server running on AWS EC2. The server is deployed as one WAR file in Tomcat 9. The client uses HTTP to submit lift rides and WebSocket connections to receive leaderboard updates.



The server does not store rides or calculate real standings. It validates requests and returns an acknowledgement. The leaderboard contains two fixed sample skiers, but each message has a new server timestamp.

Server Classes

1. **SkierServlet** handles the swipe POST endpoint. It reads the four IDs from the URL and parses the JSON body into a LiftRide object using Gson. All four IDs must be positive integers. The ride time must be between 1 and 360, and the lift ID must be between 1 and 40. Valid requests return HTTP 201, and invalid requests return HTTP 400.
2. **LiftRide** contains the time and liftID fields. ResponseMsg provides the message format used in servlet responses.
3. **HealthServlet** handles GET /health and returns HTTP 200 with {"status":"UP"}.
4. **LeaderboardEndpoint** handles /leaderboard/{resortID}. It keeps a shared ConcurrentHashMap from resort IDs to thread-safe sets of WebSocket sessions. A session is added when a client connects and removed when it closes or encounters an error. One shared scheduled executor pushes a synthetic leaderboard every 500 milliseconds.

Client Classes and Threading

SwipeClient controls the test. Its nested classes separate configuration, generated rides, HTTP attempts, final ride outcomes, subscribers, and received push measurements.

One dedicated generator thread creates all 500,000 ride records before sending starts. It puts them into an ArrayBlockingQueue. This uses more memory than generating a small batch at a time, but it means sender threads do not have to wait for the generator. Generation time is measured separately.

The warmup uses 32 sender threads. Each sends 1,000 rides and then finishes, giving 32,000 warmup rides. After warmup, a new sender pool sends the remaining 468,000 rides. I used 128 threads for the main phase.

All sender threads share one Java HttpClient, configured for HTTP/1.1 and connection reuse. A failed ride can be retried up to five times after the initial attempt. The waits between retries are 100, 200, 400, 800, and 1,600 milliseconds.

I used ten WebSocket subscribers, one for each resort. They connect before ingest starts and remain active during the test. Each subscriber records its connection establishment time, received messages, and push timestamps. If a connection drops, its coordinator attempts to reconnect using capped exponential backoff.

Measurements and Little's Law

Before the load run, the client sends 20 sequential probe requests. These are additional requests and are excluded from the 500,000 load events.

The first request took 257.689 ms, including initial connection setup. The next 19 requests had an average round-trip time of 29.729 ms using the shared client.

Using Little's Law: Estimated throughput = concurrent requests / average response time

$$\text{Warmup: } 32 / 0.029729 \approx 1{,}076.38 \text{ swipes/second}$$

$$\text{Main: } 128 / 0.029729 \approx 4{,}305.52 \text{ swipes/second}$$

These estimates assume each sender has one request in flight and that the baseline response time stays constant.

The client uses System.nanoTime() to measure elapsed request time. It buffers measurements in memory and writes CSV files after ingest finishes. This keeps file writing outside the measured sending period.

Test Results and Analysis

Test Environment

The client ran on my Mac using Microsoft OpenJDK 11.0.32.1. The server ran on an AWS EC2 t3.micro instance in us-west-2a, using Java 11 and Tomcat 9.

The server address for this run was: http://44.244.89.76:8080/assignment1

I first checked the application locally, then ran a 200-event smoke test against EC2. After that passed, I ran the full workload with 128 main sender threads and ten WebSocket subscribers.

Both submitted client directories contain the same implementation. It prints the basic metrics and exports the detailed analysis. The basic and detailed screenshots show different sections of the same full EC2 run.

Ingest Results

Metric	Result
Total load events	500,000
Successful events	500,000

Failed events	0
HTTP attempts	500,000
Retries	0
Generation time	0.035s
Warmup events	32,000
Warmup time	46.501s
Warmup throughput	688.16 swipes/s
Main-phase events	468,000
Main-phase time	203.486s
Total load time	249.991s
Overall throughput	2,000.08 swipes/s

Total load time includes warmup and the main phase. It excludes the probe requests, data generation, initial WebSocket setup, and result export.

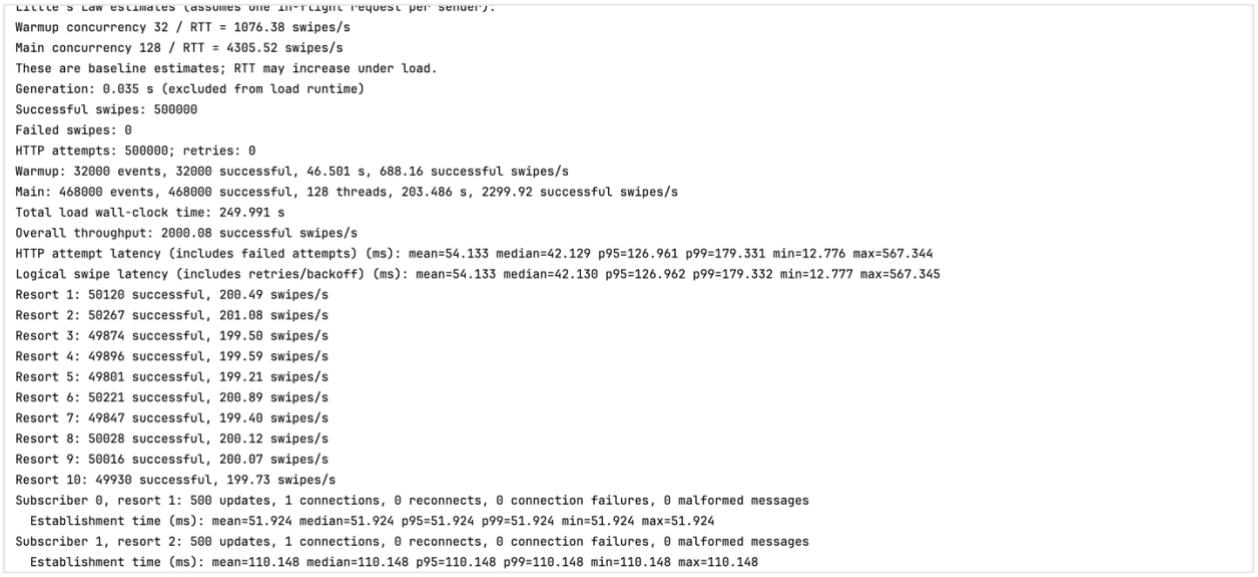


Figure 1. client basic metrics

HTTP Latency

All attempts succeeded in this run, so the following statistics cover 500,000 successful HTTP requests.

Metric	Latency (ms)
Mean	54.133
Median	42.129
p95	126.961

p99	179.331
Minimum	12.776
Maximum	567.344

Most requests completed much faster than the maximum. The difference between the median and the higher percentiles shows that some requests experienced longer delays.

Results by Resort

Per-resort throughput uses the full 249.991-second load period.

Resort	Successful swipes	Throughput (swipes/s)
1	50,120	200.49
2	50,267	201.08
3	49,874	199.50
4	49,896	199.59
5	49,801	199.21
6	50,221	200.89
7	49,847	199.40
8	50,028	200.12
9	50,016	200.07
10	49,930	199.73

The ride counts are close to evenly distributed, which is consistent with randomly selecting resorts from 1 through 10.

Predicted and Actual Throughput

The main-phase prediction was 4,305.52 swipes/s, while the measured main throughput was 2,299.92 swipes/s, about 53.4% of the prediction.

The prediction used sequential requests before the load test. Under concurrent load, the overall mean HTTP latency increased to 54.133 ms. This shows why the original response-time assumption was optimistic.

Possible contributors include network variation, scheduling overhead, connection management, and server request logging. I did not isolate these factors, so I cannot identify one of them as the confirmed bottleneck. I also did not complete a thread-count sweep, so this result does not establish that 128 threads is optimal.

Throughput over Time

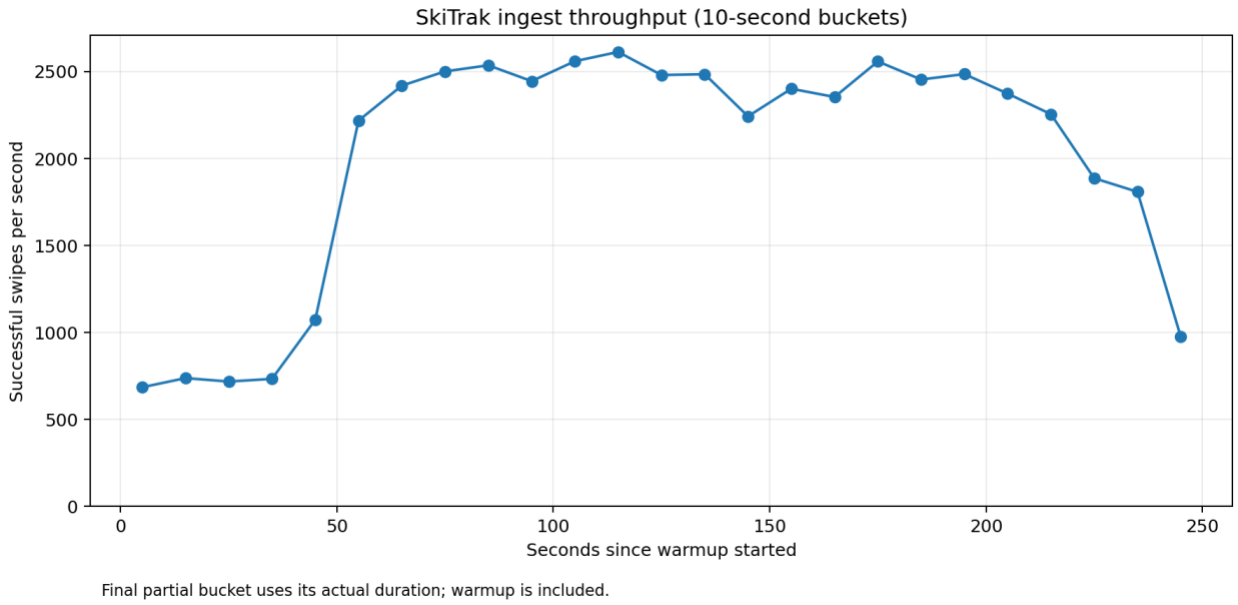


Figure 2. throughout over time

The chart groups successful completions into ten-second buckets. The final partial bucket uses its actual duration.

Throughput is lower during the 32-thread warmup and rises after the switch to 128 threads. It fluctuates during the main phase and falls near the end. As sender threads finish their assigned work, fewer requests remain in flight, which can contribute to the final decline.

WebSocket Results

All ten subscribers connected successfully. Each received 500 updates, giving 5,000 leaderboard messages in total. No reconnects, connection failures, or malformed messages were recorded.

Resort	Updates received	Connection establishment (ms)
1	500	51.924
2	500	110.148
3	500	110.041
4	500	110.271
5	500	109.798
6	500	110.298

7	500	109.172
8	500	110.106
9	500	108.871
10	500	108.201

Receiving about 500 updates per subscriber over approximately 250 seconds is consistent with the server's 500-millisecond push interval.

Push measurements were calculated as client receipt time minus the server's serverTimestamp.

Metric	Measured push-time difference (ms)
Mean	-9.169
Median	-20.199
p95	55.834
p99	83.276

These values include the clock offset between my Mac and EC2. The negative values do not represent negative network delay. I kept the original samples rather than changing them to zero. Without independently measuring the clock offset, I cannot interpret these numbers as accurate one-way network latency.

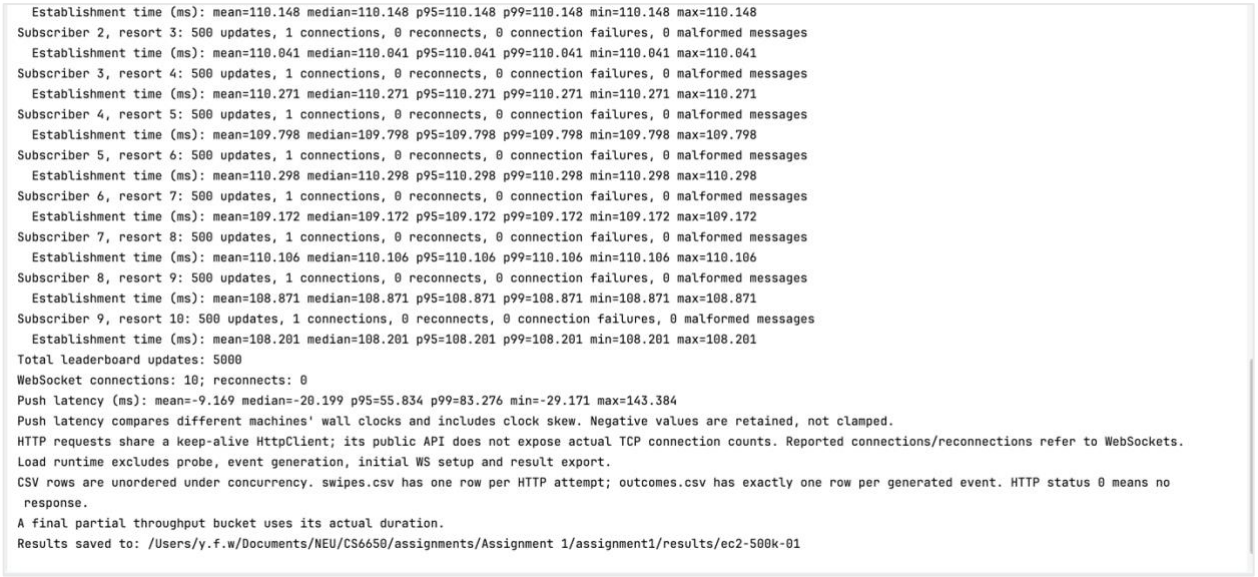


Figure 3. client latency metric

Deployment Evidence

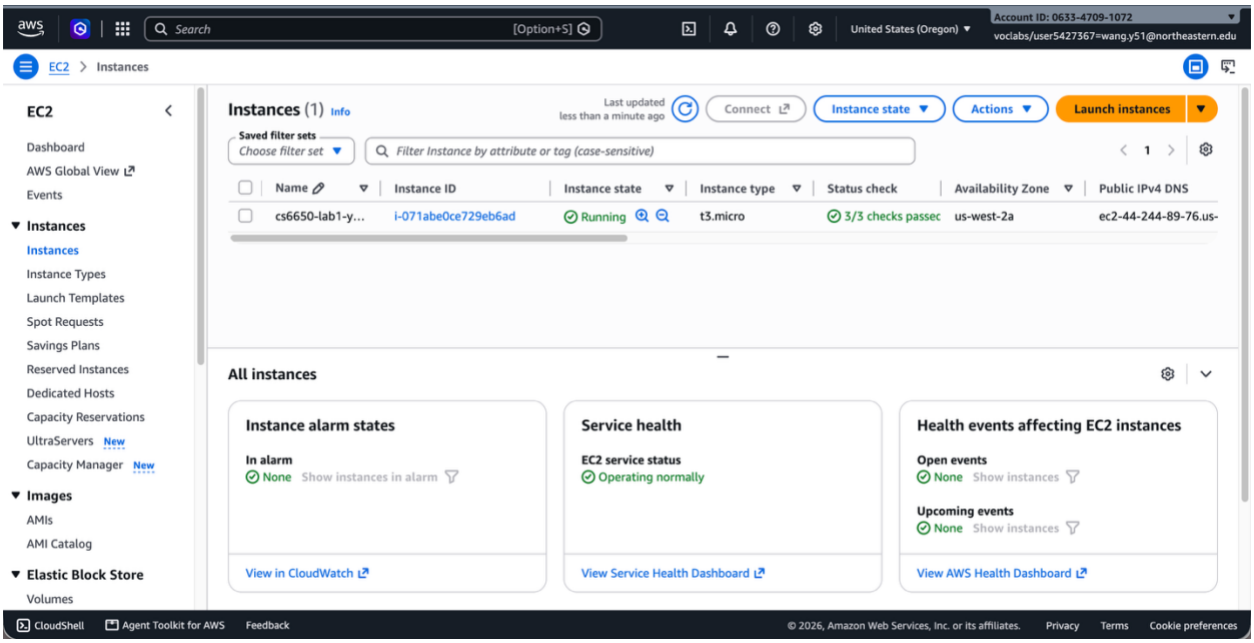


Figure 4. EC2 Console

The EC2 screenshot shows the running t3.micro instance in us-west-2a.

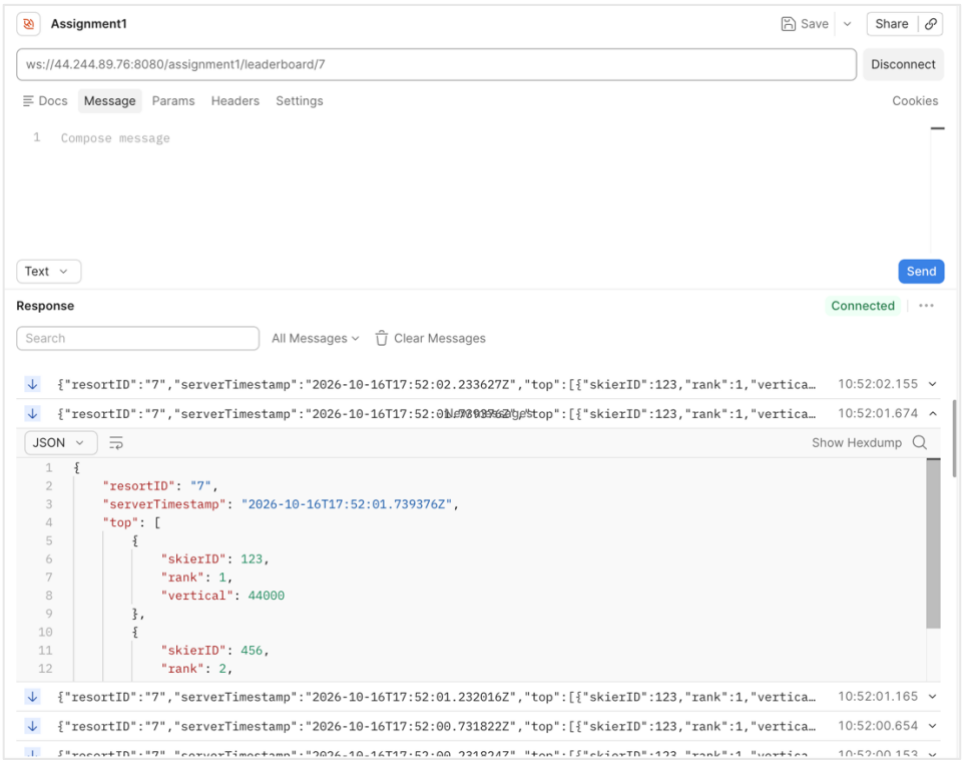


Figure 5. Postman WebSocket Connection Result

The Postman screenshot shows a connection to the EC2 leaderboard endpoint and repeated JSON messages containing the resort ID, server timestamp, and synthetic standings.

Limitations

The full run completed without failures or disconnects. It therefore does not demonstrate retry exhaustion or reconnection under a real network failure, even though the client includes those behaviors.

The connection statistics describe WebSocket connections. Java HttpClient does not expose actual HTTP TCP connection counts through its public API, so those counts are not reported.

The submitted CSV files contain the original measurements. swipes.csv records individual HTTP attempts, outcomes.csv records logical ride outcomes, and pushes.csv records received leaderboard messages.